

PRÁCTICA DE PROCESADORES DEL LENGUAJE II

Curso 2025 – 2026

Entrega de Junio

APELLIDOS Y NOMBRE: López Jiménez Álvaro

DNI: 77170267T

CENTRO ASOCIADO MATRICULADO: Almeria

CENTRO ASOCIADO DE LA SESIÓN DE CONTROL: Motril

EMAIL DE CONTACTO: alopez4433@alumno.uned.es

TELÉFONO DE CONTACTO: 654170152

¿REALIZAS LA PARTE OPCIONAL?: No

1. El analizador semántico y la comprobación de tipos

1.1. Descripción del manejo de la Tabla de Símbolos y de la Tabla de Tipos

En las clases de símbolos (SymbolVariable, SymbolConstant, SymbolParameter, SymbolFunction, SymbolProcedure) se han añadido los atributos necesarios como la dirección de memoria, el valor de las constantes, el tipo de retorno de las funciones y la lista de parámetros. En la clase de tipo TypeRecord se ha implementado el método getSize() para calcular el tamaño en memoria de cada tipo, en el que tenemos un ámbito interno donde se almacenan los campos y se van sumando para calcular el tamaño del struct siendo 1 para los tipos simples y la suma recursiva de los tamaños de sus campos para los registros.

Las comprobaciones semánticas implementadas en el parser.cup son:

- Antes de registrar cualquier símbolo o tipo se comprueba que no exista ya en el ámbito actual.
- Tipos en asignaciones: el tipo del lado derecho debe ser compatible con el del lado izquierdo.
- Tipos en expresiones: AND requiere operandos BOOLEAN; >, - y * requieren INTEGER; /= requiere operandos del mismo tipo.

- Condiciones de IF y WHILE: deben ser de tipo BOOLEAN.
- RETURN: se usa una pila de tipos de retorno para validar que el tipo devuelto coincide con el declarado en la función y que no se usa fuera de contexto.
- Paso de parámetros: se verifica el número de argumentos y la compatibilidad de tipos en la invocación de subprogramas.
- Acceso a registros: se comprueba que el identificador sea de tipo registro y que el campo accedido exista.
- Case-insensitive: todos los identificadores se convierten a mayúsculas con toUpperCase() antes de registrarlos o buscarlos.

2. Generación de código intermedio

2.1. Descripción de la estructura utilizada

Las cuádruplas se generan directamente en las reglas gramaticales y se propagan hacia arriba mediante objetos Expresion y Sentencia. La asignación de direcciones de memoria se realiza en el axioma en tres fases:

En la primera fase se asignan direcciones absolutas descendentes desde 65535 a las variables globales y temporales del ámbito de nivel 0. Los campos de registros reciben offsets relativos ascendentes desde 0, asignados en el momento de declarar el tipo en declTipo, antes de cerrar el ámbito interno.

En la segunda fase se actualizan las direcciones en las cuádruplas ya generadas, ya que los objetos Variable se crean durante el análisis antes de que el axioma asigne las direcciones definitivas. Para ello se recorren todas las cuádruplas y se actualiza la dirección de cada Variable buscando su símbolo en la tabla global.

En la tercera fase se generan las cuádruplas VARGLOBAL para inicializar todas las variables globales a 0.

Las cuádruplas implementadas son: VARGLOBAL, MV, MVA, MVP, MVPP, STP, ADD, SUB, MUL, AND, GR, NE, BR, INL, HALT, WRITESTRING, WRITEINT, WRITEBOOL, WRITELN y CADENA.

Para el acceso a campos de registros se usa MVPP (doble indirección) en lugar de MVP, ya que el temporal contiene la dirección calculada del campo, no el valor directamente. Los subprogramas tienen análisis semántico completo pero no generan código intermedio ni lanzan excepción.

3. Generación de código final

3.1. Descripción de hasta dónde se ha llegado

Se ha implementado la generación de código final completa para todo el lenguaje excepto subprogramas. El trabajo realizado ha consistido en implementar el método `translate` y el método auxiliar `operacion`.

El método `operacion` convierte cada operando a su representación en ENS2025: las variables y temporales se representan con direccionamiento directo a memoria (`/dirección`), los valores literales con direccionamiento inmediato (`#valor`) y las etiquetas con su nombre directamente.

Las principales decisiones de traducción en `translate` son:

- MVA: carga la dirección de una variable como valor inmediato usando `#`, sin el prefijo `/`.
 - MVP: usa el registro `.R0` como auxiliar para un nivel de indirección. Carga la dirección del temporal en `.R0` y lee el valor con `[.R0]`.
 - MVPP: usa `.R0` para doble indirección. Carga la dirección, lee la dirección del campo con `[.R0]` en `.R0` y lee el valor final con `[.R0]`.
 - STP: usa `.R0` para escritura indirecta. Carga la dirección del temporal destino, lee la dirección real con `[.R0]` sobrescribiendo `.R0`, y escribe el valor con `[.R0]`.
 - SUB: usa SUB o1, o2 que en ENS2025 calcula `o1 - o2` y guarda en el acumulador.
 - GR: calcula `o1 - o2` con SUB y usa BZ/BP para detectar si el resultado es cero o positivo, concluyendo que `o1 > o2` solo si el resultado es positivo.
 - AND y NE: implementados con CMP y bifurcaciones condicionales BZ/BNZ.
 - WRITEBOOL: genera código que comprueba el valor e imprime "true" o "false" usando etiquetas globales (`wbTrue`, `wbFalseStr`) definidas una sola vez al final del programa tras el HALT.
-

4. Indicaciones especiales

- La comparación entre registros con `/=` compara únicamente el primer campo del registro, no el registro completo.
- Las cadenas de texto y los literales booleanos se almacenan al final del código ensamblador como directivas DATA, después del HALT.